Image_Page

เล่มที่ 4 · AI Server

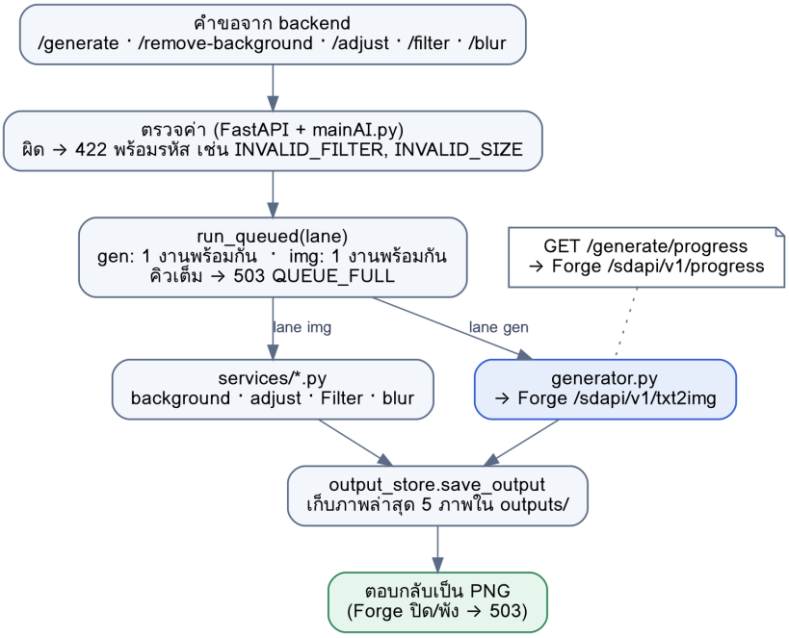
ผู้รับผิดชอบ: แก้ว

ฉบับวันที่ 13 ตุลาคม 2569

1. หน้าที่ของ AI Server

AI Server รับคำขอจาก backend อย่างเดียว ประมวลผลภาพในหน่วยความจำ แล้วตอบกลับเป็น PNG ไม่มีระบบบัญชี ไม่มีฐานข้อมูล งานสร้างภาพจากข้อความส่งต่อให้ Stable Diffusion Forge ที่รันในเครื่องเดียวกัน ส่วนงานแต่งภาพอีก 4 อย่างทำเองด้วย Python

หัวข้อ	รายละเอียด
ไฟล์หลัก	`AI_server/mainAI.py` (endpoint + คิว) และ `AI_server/services/*.py` (ตัวประมวลผล)
เทคโนโลยี	FastAPI, rembg + onnxruntime (CPU), OpenCV, Pillow, NumPy, requests
รันที่	`uvicorn mainAI:app --host 0.0.0.0 --port 8000` เครื่อง 26.84.90.188 ในวง Radmin
ต้องมีคู่กัน	Stable Diffusion Forge เปิดด้วย `--api` ที่ http://127.0.0.1:7860



ภาพที่ 1 · คำขอ 1 ครั้งเดินทางผ่าน AI Server อย่างไร

2. การติดตั้งและรัน

1. ใช้ **Python 3.12** (ห้าม 3.13: pip จะลง OpenCV 5 ที่ไม่มี `CascadeClassifier` ทำให้เบลอใบหน้าพัง)
2. สร้าง venv แล้วติดตั้งไลบรารีจาก `requirements.txt`
3. เปิด Stable Diffusion Forge ด้วยแฟล็ก `--api`
4. รัน uvicorn ในโฟลเดอร์ AI_server แล้วเปิดดู endpoint ได้ที่ http://127.0.0.1:8000/docs

```
cd AI_server
py -3.12 -m venv .venv
```

```
.venv\Scripts\activate
pip install -r requirements.txt
uvicorn mainAI:app --host 0.0.0.0 --port 8000
```

ไลบรารี	ใช้ทำอะไร
rembg, onnxruntime	ลบพื้นหลังด้วยโมเดล u2net (ต้องเป็น onnxruntime แบบ CPU ห้ามลง onnxruntime-gpu คู่กัน)
opencv-python-headless < 5	ฟิลเตอร์, ปรับสี, ตรวจจับใบหน้า
pillow, numpy	อ่าน/เขียนภาพ และคำนวณค่าสีทั้งภาพ
fastapi, uvicorn, python-multipart, pydantic	ตัวเว็บเซิร์ฟเวอร์และรับไฟล์อัปโหลด
requests	เรียก Forge

> ไฟล์ `patch.py` ถูก import เป็นอย่างแรกใน mainAI.py เพื่อเติม `np.long` ให้ NumPy 2.x ที่ไลบรารีเก่ายังเรียกอยู่ ห้ามลบ · ครั้งแรกที่ลบพื้นหลัง rembg จะดาวน์โหลดโมเดล (~170 MB) ไปไว้ที่ `C:\Users\<ชื่อ>\.u2net` จึงช้ากว่าปกติ

README ในโฟลเดอร์ AI_server ยังเขียนคำสั่งเป็น `uvicorn main:app` ซึ่งเป็นชื่อไฟล์เก่า ใช้ `mainAI:app` แทน

ค่าที่ตั้งผ่าน env

ตัวแปร	ค่าเริ่มต้น	ความหมาย
FORGE_URL	http://127.0.0.1:7860	ที่อยู่ Forge
MAX_CONCURRENT_GEN / MAX_QUEUE_GEN	1 / 10	สาย gen: ทำพร้อมกัน / รอคิวได้ (ต้องตรงกับ backend)
MAX_CONCURRENT_IMG / MAX_QUEUE_IMG	1 / 10	สาย img: ทำพร้อมกัน / รอคิวได้ (ต้องตรงกับ backend)
REMBG_MODEL	u2net	โมเดลลบพื้นหลัง
OUTPUT_DIR / MAX_OUTPUTS	AI_server/outputs / 5	โฟลเดอร์และจำนวนภาพผลลัพธ์ล่าสุดที่เก็บไว้ดูย้อนหลัง

3. รายการ endpoint

Method	Path	รับ	error ที่ตอบได้
POST	/generate	JSON: prompt (1–2000 ตัว), negative_prompt, seed (−1 = สุ่ม), size, lora	422 INVALID_PROMPT / INVALID_SIZE / INVALID_LORA · 503 Forge ปิดหรือพัง

Method	Path	รับ	error ที่ตอบได้
GET	/generate/progress	—	Forge ไม่ตอบ → {"progress": null}
POST	/remove-background	image, output (transparent / white / black), refine_edge	422 INVALID_IMAGE
POST	/adjust	image, brightness, contrast, saturation, temperature, vignette (−100..100)	422 INVALID_ADJUST_REQUEST
POST	/filter	image, filter_name, intensity (−100..100)	422 INVALID_FILTER / INVALID_INTENSITY / INVALID_IMAGE
POST	/blur	image, blur_type (gaussian / background / face), blur_amount (0..100)	422 INVALID_BLUR_TYPE / INVALID_BLUR_AMOUNT / INVALID_IMAGE
GET	/queue	—	จำนวนงานในแต่ละสาย
GET	/health	—	{"status": "ok"}

ทุก endpoint ที่ประมวลผลภาพตอบ `image/png` และผ่าน `run_queued` ถ้าคิวของสายเต็มตอบ 503 QUEUE_FULL

4. ระบบคิวฝั่ง AI (run_queued)

AI มีคิว 2 สายเหมือน backend เพื่อกันเครื่องรับงานหนักพร้อมกันเกินแรม (เครื่องมีแรม 16 GB)
งานในสายเดียวกันทำทีละงาน งานจริงรันใน thread pool ไม่บล็อกคำขออื่น เช่น การถาม % ความคืบหน้า

สาย	endpoint	ทำพร้อมกัน	รอได้
gen	/generate	1	10
img	/remove-background, /adjust, /filter, /blur	1	10

หลังทำเสร็จทุกครั้ง `_run_and_save` เรียก `save_output` เก็บภาพลงโฟลเดอร์ outputs (เก็บไว้ 5 ภาพล่าสุด)
ถ้าบันทึกไม่ได้จะเขียน log อย่างเดียว ไม่ทำให้คำขอพัง

5. ตัวประมวลผลแต่ละตัว (services)

5.1 generator.py — สร้างภาพจากข้อความ

- ส่ง `POST {FORGE_URL}/sdapi/v1/txt2img` ด้วย prompt, negative_prompt, seed, width, height (timeout 300 วินาที) แล้วถอด base64 ของภาพแรกกลับเป็น PNG

- ขนาดที่รองรับ (IMAGE_SIZES): 512x512, 512x768 (ค่าเริ่มต้น), 768x512, 768x768
- LoRA ต่อท้าย prompt เป็น `<lora:ชื่อไฟล์:น้ำหนัก>` พร้อมคำกระตุ้น (trigger) ถ้ามี
- error ฝั่ง Forge (ต่อไม่ติด, ตอบ HTTP error, ตอบผิดรูปแบบ) → RuntimeError → 503 พร้อมข้อความภาษาไทย รายละเอียดจริงอยู่ใน log เท่านั้น
- `get_progress()` ถาม `GET /sdapi/v1/progress?skip_current_image=true` → progress 0–1, เวลาที่เหลือ, step ปัจจุบัน / ทั้งหมด

LoRA (ชื่อที่หน้าเว็บส่ง)	ไฟล์ใน models/Lora	trigger
pixel_art	pixelbuildings128-v2	—
outline	outline_xl_kohaku_delta_spv5x	—
linear_manga	LineAniRedmondV2-Lineart-LineAniAF	lineart, LineAniAF

5.2 background.py — ลบพื้นหลัง

- ใช้ `rembg.remove` กับ session โมเดล u2net ที่โหลดครั้งเดียวแล้วใช้ซ้ำ
- `refine_edge`: หดขอบ alpha 1 พิกเซลแล้วเบลอเล็กน้อย ขอบจึงไม่แข็ง
- `output`: transparent = PNG โปร่งใส · white / black = วางบนพื้นสีนั้นแล้วตัดช่องโปร่งใสออก

5.3 adjust.py — ปรับแสงสี

ทุกค่ามีช่วง −100 ถึง +100 โดย 0 = ไม่เปลี่ยน ค่าที่เป็น 0 จะถูกข้าม ภาพจึงไม่เปลี่ยนโดยไม่จำเป็น ปรับตามลำดับนี้

ลำดับ	ค่า	วิธีคำนวณ
1	temperature	ดันช่องแดงกับน้ำเงินสวนทางกัน สูงสุด ±20%
2	saturation	ผสมระหว่างภาพเทากับภาพเดิม (−100 = ขาวดำ, +100 = สีจัด 2 เท่า)
3	contrast	ดึงค่าสีออก/เข้าหาเทากลาง 128 (−100 = เทาล้วน)
4	brightness	บวก/ลบค่าสีทุกพิกเซลเท่ากัน สูงสุด ±100
5	vignette	ยิ่งไกลกลางภาพยิ่งมืด (+) หรือสว่าง (−) สูงสุด 60% ที่ขอบ

5.4 Filter.py — โทนสีสำเร็จรูป

มี 10 ฟิลเตอร์: grayscale, sepia, warm, cool, vivid, vintage, cinematic, sharpen, sketch, cartoon ความเข้มคำนวณจาก `ผลลัพธ์ = ต้นฉบับ + k × (ภาพฟิลเตอร์ − ต้นฉบับ)` โดย k = intensity / 100 ค่า 0 ได้ภาพเดิม ค่า 100 ได้ฟิลเตอร์เต็ม และค่าติดลบจะได้ผลด้านตรงข้าม (เช่น sharpen ติดลบ = เบลอ)

หน้าเว็บให้เลือกความเข้มแค่ 0–100% แต่ AI รองรับ −100 ถึง 100

5.5 blur.py — เบลอ

blur_type	วิธีทำ
gaussian	Gaussian blur ทั้งภาพ รัศมีสูงสุด 25 พิกเซลที่ blur_amount 100
background	ใช้ rembg หาวัตถุหลัก แล้วเบลอเฉพาะส่วนที่ไม่ใช่วัตถุ
face	หาใบหน้าด้วย Haar cascade ของ OpenCV (ย่อภาพไม่เกิน 640 พิกเซลก่อนหา) แล้วเบลอเป็นวงรีขอบนุ่ม · ไม่เจอใบหน้า = คืนภาพเดิม

blur_amount = 0 คืนภาพเดิมทันทีโดยไม่ประมวลผล

5.6 output_store.py — เก็บภาพผลลัพธ์

เก็บภาพผลลัพธ์ล่าสุดไว้ใน `AI_server/outputs` ตั้งชื่อเป็น `วันเวลา_ชื่องาน_เลขสุ่ม.png` เกิน 5 ภาพลบอันเก่าทิ้ง เขียนเป็นไฟล์ .tmp ก่อนแล้วค่อยเปลี่ยนชื่อ กันไฟล์ครึ่ง ๆ กลาง ๆ

ปัญหาที่รู้อยู่: ถ้าบันทึกหลายภาพในวินาทีเดียวกัน ลำดับ "เก่า-ใหม่" ตัดสินจากเลขสุ่ม จึงอาจลบภาพที่ใหม่กว่าแทนทีมตัดสินใจปล่อยไว้ เพราะงานจริงไม่ค่อยเสร็จพร้อมกัน

6. การใช้แรมและข้อควรระวัง

- Forge และโมเดล rembg กินแรมมาก การตั้งให้แต่ละสายทำทีละ 1 งานช่วยไม่ให้แรม 16 GB เต็มจน Windows ไปใช้ pagefile
- ถ้าจะเพิ่มจำนวนงานพร้อมกัน ต้องแก้ทั้ง `MAX_CONCURRENT_*` ฝั่ง AI และ `*_CONCURRENCY` ฝั่ง backend ให้เท่ากัน
- เปลี่ยนรายการ LoRA หรือขนาดภาพ ต้องแก้ให้ตรงกันทั้ง `generator.py`, `mainBackend.py` และ `config.js` (มีเทสใน test_generate_options.py คอยเช็ค)
- ข้อความ error ที่ส่งกลับไม่มี URL หรือรายละเอียดภายใน ดูสาเหตุจริงได้จาก log ของเครื่อง AI

7. การทดสอบที่เกี่ยวกับ AI

ไฟล์เทส	สิ่งที่เช็ค
test_image_tools.py	Adjust / Filter / Blur: ค่าจากหน้าเว็บถึงฟังก์ชันครบ, ค่าผิดได้ 422, ฟังก์ชันจริงให้ผลถูก (เช่น saturation −100 ได้ขาวดำ, เบลอแล้วขอบนุ่มลง, ไม่มีหน้า = ภาพเดิม)
test_generate_frontend_backend_ai.py	prompt ไปถึง Forge, ภาพ 512×768, Forge ปิด / พัง / ตอบผิดรูปแบบ แล้วได้ 503 พร้อมข้อความที่ไม่เปิดเผยรายละเอียดภายใน
test_generate_options.py	LoRA ต่อท้าย prompt ถูก, ขนาดกลายเป็น width/height ถูก, ค่าที่ไม่รองรับไม่ถึง Forge

ไฟล์เทส	สิ่งที่เช็ค
test_remove_background_frontend_backend_ai.py	ตัวเลือก output / refine_edge ถึง AI, เทสโมเดลจริง (ข้ามได้ด้วย -m "not model")
test_progress.py	/generate/progress อ่าน % จาก Forge, Forge ปิดได้ progress = null
test_proxy_and_outputs.py	output_store เก็บไม่เกิน 5 ภาพ, ชื่อไม่ชน, ชื่องานแปลก ๆ ไม่หลุดออกนอกโฟลเดอร์, บันทึกไม่ได้แล้วไม่พัง